

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет прикладной математики, информатики и механики

Кафедра математического обеспечения ЭВМ

**Разработка программы визуализации фазовых портретов двумерных
систем дифференциальных уравнений на OpenGL**

Курсовая работа

01.03.02 Прикладная математика и информатика

Зав. кафедрой	____к.ф.-м.н, доц. Каплиева Н.А.
Обучающийся	____ Хмель И. А.
Руководитель	____ к.ф.-м.н, доц. Трофименко Е. В.

Воронеж 2026

Содержание

Введение.....	3
1. Обзор предметной области	4
1.1. Двумерные автономные системы ОДУ и фазовый портрет	4
1.2. Обзор аналогов и обоснование выбора технологий	5
2. Постановка и анализ задачи.....	6
2.1. Анализ задачи и математические основы.....	7
1. Поле направлений и геометрическая интерпретация решений	7
2. Численное интегрирование траекторий методом Рунге-Кутты 4-го порядка	8
3. Точки равновесия и классификация по Якобиану.....	9
3. Средства реализации и минимальные системные требования	11
3.1 Средства реализации	11
3.2. Минимальные системные требования	12
4. Проектирование и реализация	13
4.1. Архитектура и структура	13
4.2. Организация данных	15
4.3. Численные методы	16
4.4. Реализация систем	18
4.5. Камера, координаты мира и управление	19
4.6. Подсистема визуализации.....	20
4.7. Шейдеры и создание программы OpenGL.....	21
4.8. Главный файл приложения и цикл работы.....	21
5. Вычислительный эксперимент и демонстрация работы	24
5.1. Линейные системы	24
5.2. Осциллятор Ван-дер-Поля	26
4.3. Модель Лотки-Вольтерра.....	26
4.4. Пользовательский ввод	27
Заключение	29
Список использованных источников.....	30
Приложение	31

Введение

Системы дифференциальных уравнений первого порядка широко применяются для описания динамики физических, биологических и технических процессов. В реальных задачах часто необходимо не столько получить аналитическое выражение решения, сколько понять качественное поведение системы: устойчивость состояний, наличие колебаний, предельных циклов, областей притяжения и разделяющих траекторий (сепаратрис).

Для двумерных автономных систем вида

$$\dot{x} = f(x, y), \dot{y} = g(x, y)$$

наиболее наглядным инструментом исследования является фазовый портрет, включающий поле направлений, траектории, точки равновесия и их типы. Для нелинейных систем аналитическое решение зачастую отсутствует, поэтому используются численные методы и компьютерная визуализация.

Целью курсовой работы является разработка программы, предназначенной для интерактивной визуализации фазовых портретов двумерных систем дифференциальных уравнений на основе графического API OpenGL.

1. Обзор предметной области

1.1. Двумерные автономные системы ОДУ и фазовый портрет

Рассматривается система дифференциальных уравнений первого порядка:

$$\frac{\partial x}{\partial t} = f(x, y), \quad \frac{\partial y}{\partial t} = g(x, y).$$

Удобно объединить переменные в вектор состояния $p(t) = (x(t), y(t))$. Тогда система записывается в векторном виде:

$$\dot{p} = F(p), \quad F(x, y) = (f(x, y), g(x, y)).$$

Так как система автономная (не зависит явно от времени), в каждой точке фазовой плоскости (x, y) задаётся вектор скорости $F(x, y)$. То есть система задаёт на плоскости векторное поле. Траектории решений являются интегральными кривыми этого векторного поля.

Фазовый портрет двумерной системы – это совокупность (рис. 1):

- поля направлений (векторное поле).
- набора траекторий из различных начальных условий.
- точек равновесия (стационарные точки) и их классификации.
- характерных кривых, разделяющих области с различным поведением решений.

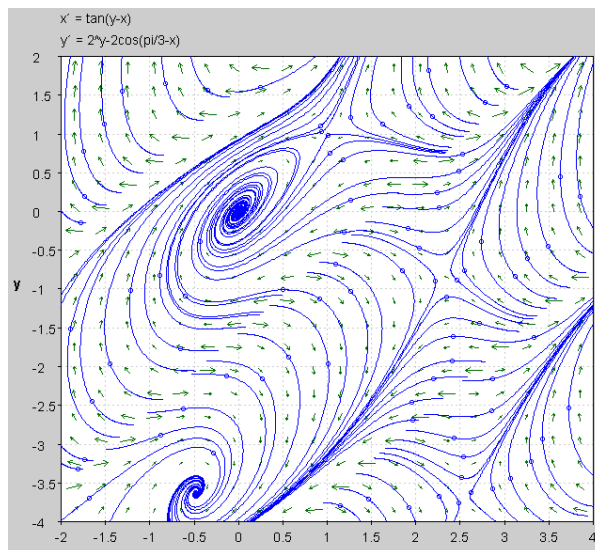


Рис. 1. Пример визуализации фазового портрета

1.2. Обзор аналогов и обоснование выбора технологий

Аналоги визуализации фазовых портретов встречаются в математических пакетах и учебных инструментах, например в MATLAB. Это платформа для программирования и численных вычислений. Включает высокоуровневый интерпретируемый язык программирования, пакет прикладных программ и интегрированную среду разработки. Этот инструмент также имеет при себе широкий спектр функций, например, для анализа данных в различных областях математики, линейной алгебры или фазового анализа (рис. 2). Однако MATLAB, как и другие похожие пакеты ПО – это зачастую тяжелая, серьезная программа, требующая подготовки и большого багажа знаний. Для учебных и исследовательских же задач полезно иметь простое, легковесное приложение с возможностью исследовать известные классические системы или вводить собственные уравнения без тяжёлых зависимостей.

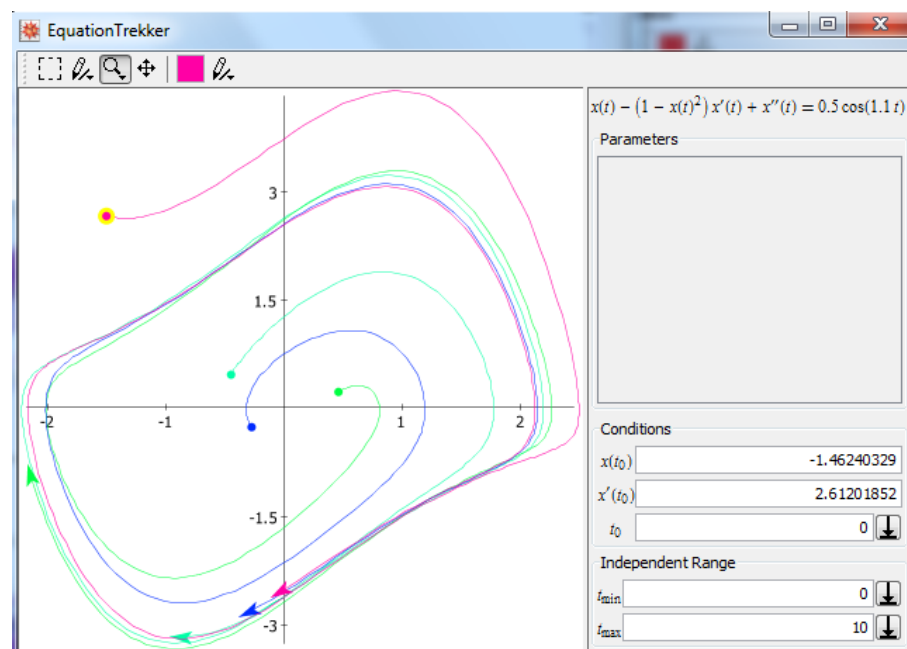


Рис. 2. Пример фазового портрета в инструменте MATLAB

2. Постановка и анализ задачи

Целью данной работы является разработка программы, предназначенной для интерактивной визуализации фазовых портретов двумерных систем дифференциальных уравнений на основе графического API OpenGL. Для достижения цели были поставлены следующие задачи:

1. Изучить математические основы фазового анализа двумерных автономных систем.
2. Реализовать построение поля направлений и численное интегрирование траекторий.
3. Реализовать поиск и классификацию точек равновесия по матрице Якоби.
4. Задать пользовательский ввод правых частей системы и параметров.
5. Разработать графическую подсистему отрисовки и интерфейс управления.

Разрабатываемая программа должна обеспечивать интерактивное исследование двумерных систем ОДУ. В ходе анализа требований сформулированы функциональные возможности:

1. Выбор системы из набора встроенных примеров (линейная система, Ван-дер-Поль, Лотка–Вольтерра и т.д.).
2. Пользовательский режим (Custom): ввод $f(x,y)$ и $g(x,y)$ в виде выражений, содержащих переменные x и y , а также параметры.
3. Настройка параметров системы и численного интегрирования (шаг, число шагов, область визуализации).
4. Фоновое интегрирование, линеаризация и сэмплирование.
5. Построение поля направлений на сетке.
6. Интерактивное добавление траекторий кликом по фазовой плоскости и режим Demo-сетки.
7. Поиск точек равновесия и отображение их типа.
8. Графический интерфейс управления и подсказки по вводу функций.

Нефункциональные требования:

- достаточная производительность при отрисовке большого числа векторных сегментов;
- устойчивость к ошибкам ввода формул (диагностика и отказ от пересчёта при ошибке);
- удобство управления (UI, масштабирование/перемещение вида).

2.1. Анализ задачи и математические основы

1. Поле направлений и геометрическая интерпретация решений

Вектор скорости $F(x,y)$ определяет направление движения решения в фазовой плоскости. Для построения поля направлений выбирается прямоугольная область $[x_{min}, x_{max}] \times [y_{min}, y_{max}]$ и сетка $N_x \times N_y$. В каждой узловой точке сетки вычисляется вектор $v = F(p)$. Чтобы визуализация была читаемой, направление нормируется:

$$\hat{v} = \frac{v}{\|v\|},$$

а на экране рисуется короткий отрезок фиксированной длины, ориентированный по $\hat{v}$. Модуль $\|v\|$ может отображаться цветом, что помогает увидеть области высокой или низкой скорости (рис. 3).

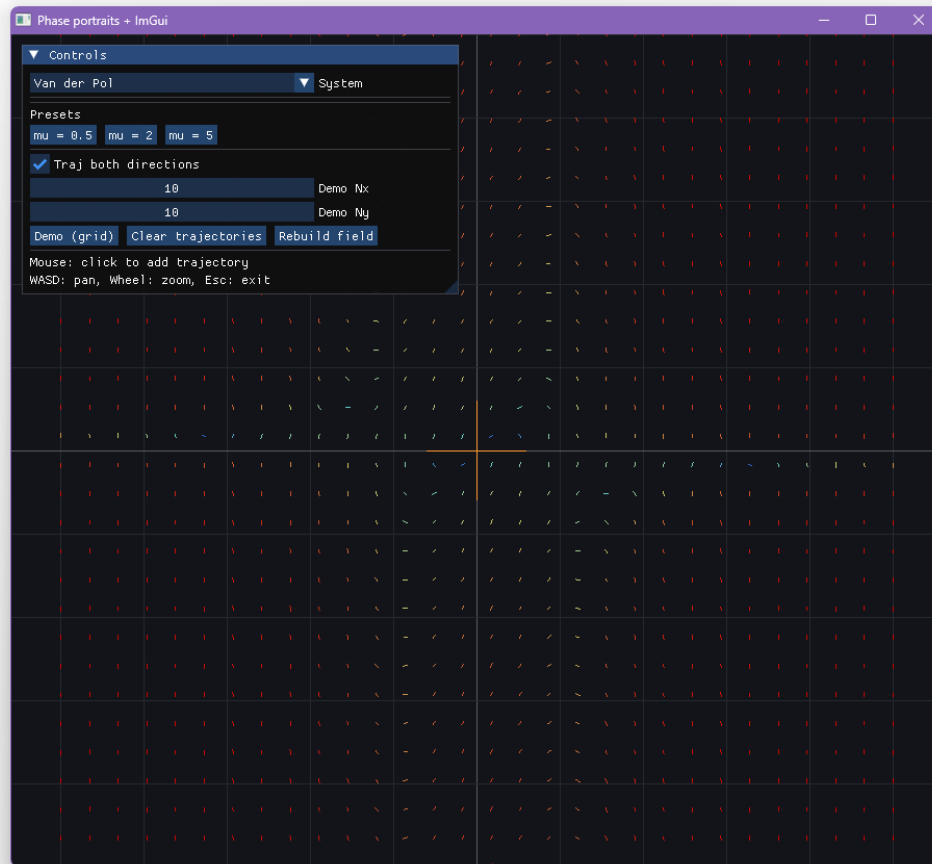


Рис. 3. Поле направлений на сетке в выбранной области фазовой плоскости

2. Численное интегрирование траекторий методом Рунге-Кутты 4-го порядка

Траектория решения - это кривая на плоскости (x,y) , по которой движется решение при изменении времени. Она строится как численное решение задачи Коши:

$$\dot{p} = F(p), \quad p(0) = p_0.$$

Поскольку метод Эйлера слишком груб, для обеспечения баланса точности и эффективности был применен метод Рунге-Кутты 4-го порядка. Так, один шаг интегрирования с шагом h вычисляется по следующим формулам:

$$k_1 = F(p_n)$$

$$k_2 = F\left(p_n + \frac{h}{2}k_1\right)$$

$$k_3 = F\left(p_n + \frac{h}{2}k_2\right)$$

$$k_4 = F\left(p_n + \frac{h}{2}k_3\right)$$

$$p_{n+1} = p_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$

где k – оценки наклона решения на разных этапах шага интегрирования, p_n – приближение решения на текущем шаге, p_{n+1} – на следующем.

Вместо одного наклона, как в Эйлере, берётся среднее по шагу, что сильно повышает точность и стабильность.

Процесс повторяется до выполнения критерия остановки: выход за пределы визуализируемой области, достижение ограничения по числу шагов, либо попадание в окрестность равновесия. Эти ограничения необходимы для устойчивости и предсказуемости построения траекторий в разрабатываемой интерактивной программе.

3. Точки равновесия и классификация по Якобиану

Точка равновесия, или стационарная точка $\dot{p}$ удовлетворяет условию:

$$F(\dot{p}) = 0,$$

то есть обе производные по времени равны нулю. Для анализа поведения решений в окрестности равновесия применяется линеаризация. Матрица Якоби определяется как:

$$J(p) = \begin{pmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{pmatrix}.$$

В пользовательском режиме аналитические производные недоступны, поэтому элементы J вычисляются численным методом конечных разностей:

$$\frac{\partial f}{\partial x}(p) \approx \frac{f(x + \varepsilon, y) - f(x - \varepsilon, y)}{2\varepsilon},$$

где ε – окрестность равновесия. Аналогично вычисления проводятся для остальных производных. Тип равновесия определяется собственными

значениями матрицы $J(\dot{p})$, достаточно воспользоваться следом, определителем и дискриминантом. Можно выделить некоторые ключевые случаи (рис. 4):

- седло: действительные собственные значения разных знаков;
- устойчивый узел: действительные части собственных значений отрицательны;
- неустойчивый узел: действительные части положительные.

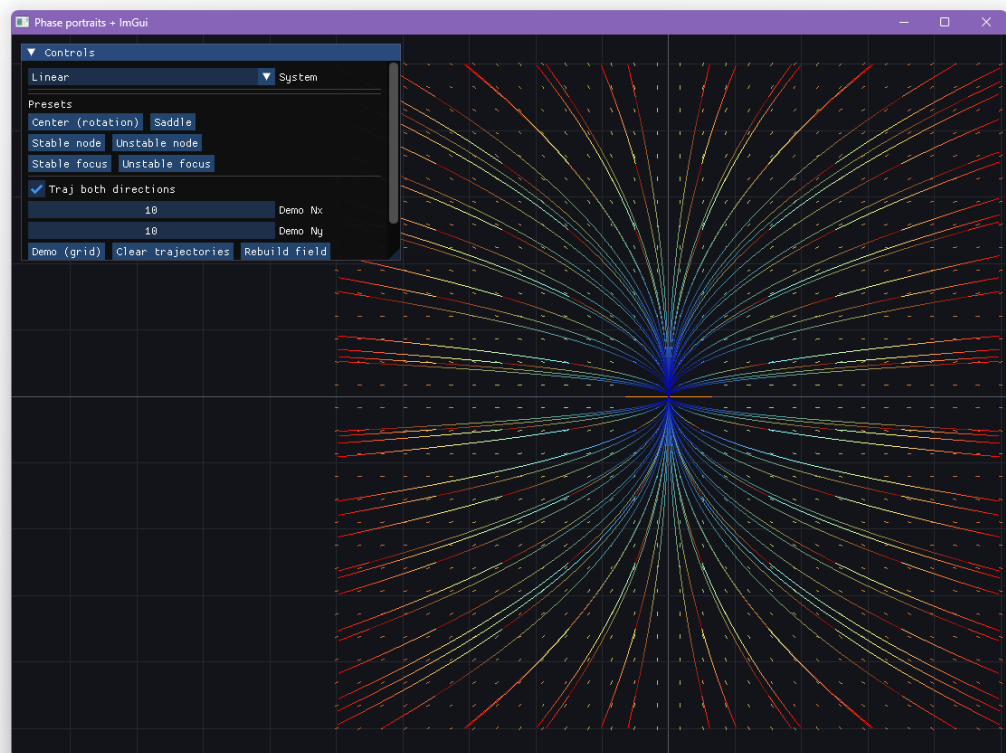


Рис. 4. Устойчивый узел

3. Средства реализации и минимальные системные требования

3.1 Средства реализации

Для создания программы был выбран язык программирования C++, среда программирования Visual Studio, а также набор технологий реализации:

- OpenGL 4.0 – кроссплатформенный графический API, описывающий набор функций и их точное поведение для написания приложений, использующих двумерную и трёхмерную компьютерную графику.
- GLFW – библиотека с открытым исходным кодом для кроссплатформенной разработки графических приложений. GLFW подходит для проектов, где требуется низкоуровневое управление графическими ресурсами, такими как создание окон, контексты рендеринга и обработка ввода с клавиатуры, мыши и других устройств.
- GLAD – открытая библиотека, которая упрощает процесс загрузки и использования функций OpenGL. Она генерирует необходимый код для загрузки указателей функций OpenGL во время выполнения, устраняя необходимость загружать функции вручную.
- Dear ImGui – это легковесная библиотека для создания графических пользовательских интерфейсов (GUI) на языке C++. Она ориентирована на простоту и продуктивность, позволяет быстро создавать инструменты для создания контента, визуализации и отладки.
- TinyExpr – компактный рекурсивный парсер и движок оценки математических выражений. TinyExpr поддерживает стандартные математические операторы, функции и привязку переменных во время выполнения.
- GLM – заголовочная библиотека на C++, спроектированная для решения математических задач в графических приложениях. Предоставляет удобные типы данных и эффективные алгоритмы для работы с векторами и матрицами.

3.2. Минимальные системные требования

Для запуска программы требуется:

- ОС: Windows 7/8.1/10/11;
- видеокарта с поддержкой OpenGL 4.0;
- CPU: любой современный, так как основная нагрузка – расчёты и линии;
- ОЗУ: от 2 ГБ (фактически достаточно меньше);
- среда разработки: Visual Studio 2026

4. Проектирование и реализация

При реализации программы была создана диаграмма вариантов использования, изображённая на рис. 9.



Рис. 9. Use case диаграмма

4.1. Архитектура и структура

Программа реализована на языке C++ и использует набор внешних библиотек: GLAD, GLFW, GLM, Dear ImGui, tinyexpr. Данные библиотеки лежат в папке packages внутри проекта (рис. 6) и подключены через свойства конфигурации Visual Studio. Исходный код проекта организован разбиением на функциональные модули в виде отдельных файлов (рис. 5):

1. `kursovaya.cpp`. Является точкой входа в программу. Здесь происходит инициализация графики, графического контекста и всех меню приложения.
2. `camera2d.h`. Реализация двухплоскостной камеры: преобразования координат, матрица вида, проекции, отображение «мира».
3. `systems.h`. Это набор вшитых в программу систем ОДУ и единый интерфейс вычисления $F(x, y)$. Нужен для базовой демонстрации работы программы.

4. custom_system.h. Реализация сторонних систем ОДУ, введенных пользователем. Точечная настройка всех коэффициентов и компиляция выражений с помощью парсера tinyexpr.
5. phase_math.h. Численная математика: метод Рунге-Кутты, равновесия, Якобиан и вспомогательные функции.
6. renderer_2d.h. Дает логику отрисовки линий, хранение вершин и работу с видеобуферами VBO и VAO (вершины и их формат).
7. shaders.h. Исходники с управлением шейдерами и другими параметрами.
8. gl_utils.h. Вспомогательные функции OpenGL: проверка ошибок, утилиты для компиляции и линковки.

Такое разделение приносит удобство в работе с математической моделью, графикой, управлением и интерфейсом.

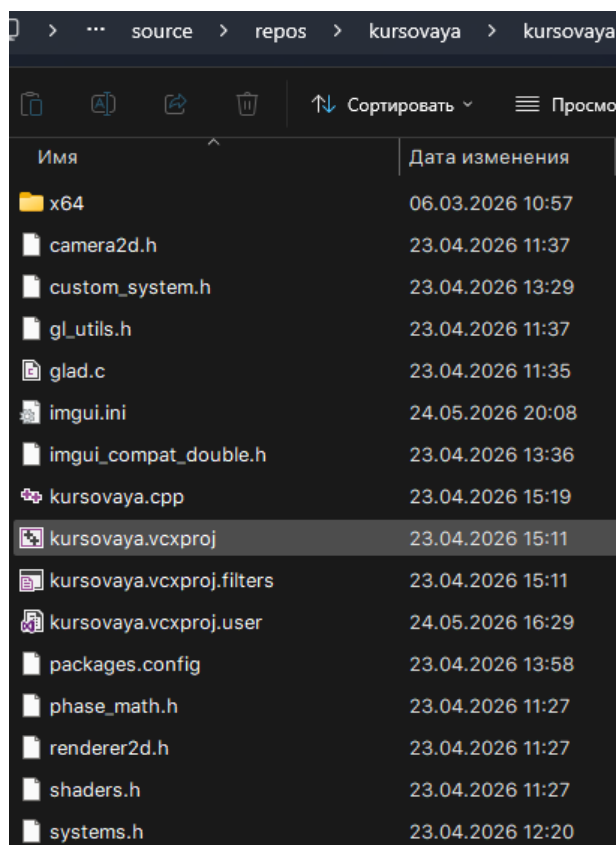


Рис. 5. Основные файлы программы

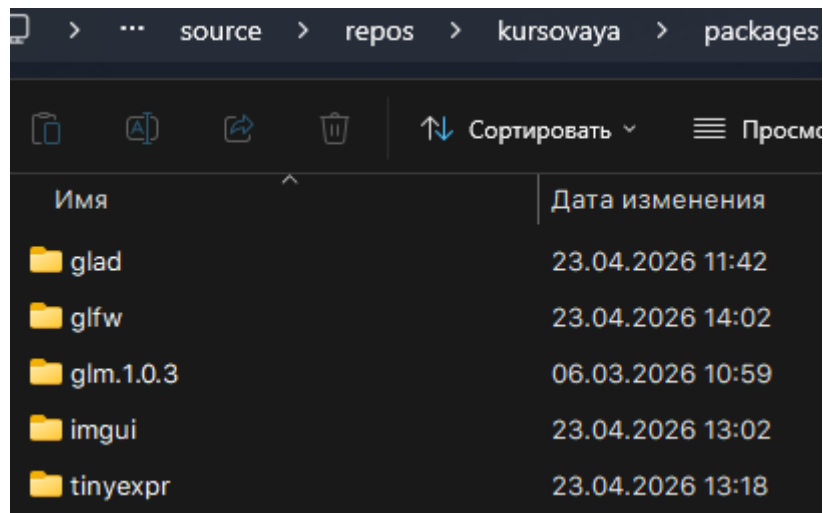


Рис. 6. Внешние библиотеки

4.2. Организация данных

Для интерактивного приложения важна единая структура состояния, в которой хранится как текущая математическая модель, так и данные для визуализации и интерфейса. В файле `kursovaya.cpp` введена структура `AppState`, содержащая:

1. Камера и область мира: `Camera2D cam` – параметры вида (центр, масштаб, размеры), `phase::AABB worldbox` – границы области, внутри которой строятся поле, траектории и ведется поиск равновесий.
2. Экземпляры систем: `phase::LinearSystem linear`, `phase::VanDerPol vdp`, `phase::LotkaVolterra lotka`, `phase::CustomSystem custom`. Выбор активной системы задаётся перечислением `enum class SystemKind {Linear, VanDerPol, Lotka, Custom}` и функцией `const phase::ISystem2D& sys() const`, которая возвращает ссылку на текущую систему, т.е. все вычисления выполняются через единый интерфейс `ISystem2D::F(p)`.
3. Настройки траектории: `phase::TrajectorySettings trajS`; Включает шаг интегрирования `h`, ограничение `maxSteps`, ограничение скачка `maxDelta`, критерий остановки `stopSpeedEps`, а также направление по времени (`Forward/Backward`).

4. Данные для отрисовки (батчи): *PolylineBatch trajectories*; – набор полилиний траекторий (несколько GL_LINE_STRIP в одном VBO); *LinesBatch vectorField*; – поле направлений (много сегментов GL_LINES); *LinesBatch gridAxes*; – сетка и оси; *LinesBatch equilibriaCrosses*; – крестики равновесий.
5. OpenGL: *GLuint program*; – шейдерная программа; *GLint uVP*; – uniform-локация матрицы VP.
6. Результаты анализа равновесий: *std::vector<phase::EquilibriumInfo> eqInfos*; хранит точку, Якобиан, след, определитель, дискриминант и классификацию.
7. UI: параметры демо-сетки: *demoNx*, *demoNy*; режим “в обе стороны времени”: *bothDirections*; буферы для Custom-ввода: *fxBuf*, *fyBuf* (так как ImGui InputText работает с char[]); *customError* – текст ошибки компиляции tinyexpr.

Таким образом, AppState представляет собой контейнер-структуру со всем, что нужно программе в кадре: состояние, вычисления и геометрия.

4.3. Численные методы

Файл *phase_math.h* задаёт базовую модель двумерной автономной системы:

$$\dot{p} = F(p), p = (x, y)$$

Вводится интерфейс системы ISystem2D:

```
struct ISystem2D {
    virtual ~ISystem2D() = default;
    virtual Vec2 F(Vec2 p) const = 0;
    virtual const char* name() const = 0;
};
```

Это позволяет использовать одинаковые алгоритмы независимо от того, является ли система встроенной или пользовательской.

Интегрирование реализовано в структуре RK4, метод *step()* вычисляет один шаг:

$$p_{n+1} = p_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$

где k_i вычисляются через вызовы *sys->F(...)* в промежуточных точках.

Функция *Trajectory integrateTrajectory(const ISystem2D& sys, Vec2 p0, const TrajectorySettings& s)* строит массив точек траектории в цикле до *maxSteps*. Остановка происходит при выполнении любого из условий:

- $|F(p)| < stopSpeedEps$ – достигнута окрестность равновесия;
- точка стала нечисловой (NaN/Inf);
- слишком большой шаг $delta > maxDelta$;
- выход за границы *bounds*.

Поле направлений строится через *std::vector<FieldSample> sampleVectorField(sys, box, nx, ny)*. Функция проходит двойным циклом по сетке, для каждой точки возвращает пару $(p, v=F(p))$. Нормировка и рисование выполняются уже на уровне *kursovaya.cpp*.

Якобиан вычисляется функцией *Mat2 jacobianCentralDiff(const ISystem2D& sys, Vec2 p, double h)* методом конечных разностей. Далее вычисляются инварианты 2 на 2: след *tr*, определитель *det* и дискриминант *disc*. Функция *classifyByTrDet(...)* выдаёт тип равновесия: седло, устойчивый или неустойчивый узел, фокус, центр, вырожденный случай.

Поиск равновесий реализован функцией *findEquilibriaGridNewton(sys, box, nx, ny, seedSpeedThresh, mergeDist)*. Алгоритм следующий:

1. грубо пройти по сетке $n_x \times n_y$;
2. отобрать кандидатов по условию $|F(p)| < stopSpeedThresh$;
3. уточнить каждый кандидат методом Ньютона *newtonSolveEquilibrium*;
4. удалить близкие дубликаты по *mergeDist*.

4.4. Реализация систем

Файл `systems.h` содержит реализации трёх систем, каждая из которых наследует `ISystem2D` и реализует $F(p)$:

1. Линейная система 2×2 : $\dot{x} = ax + by, \dot{y} = cx + dy$. Параметры a, b, c, d задаются из UI;
2. Ван-дер-Поль: $\dot{x} = y, \dot{y} = \mu(1 - x^2)y - x$.
3. Лотка-Вольтерра: $\dot{x} = ax - bxy, \dot{y} = -cy + dxy$.

`CustomSystem` позволяет задавать $f(x,y)$ и $g(x,y)$ строками. Внутри хранятся:

- строки $fxStr, fyStr$;
- параметры a,b,c,d,e,k,m,n,p,q ;
- переменные $xVar, yVar$, на которые ссылается `tinyexpr`;
- `compiled`-выражения $te_expr * fx, fy$;
- таблица `vars`.

Компиляция выполняется методом `bool compile(newFx, newFy, outError)`. Он вызывает `te_compile` для обеих формул и формирует диагностическое сообщение при ошибке (позиция ошибки). Вычисление выполняется в `Vec2 F(Vec2 p) const`, где в $xVar/yVar$ записываются текущие x, y , затем вызываются `te_eval(fx)` и `te_eval(fy)`, и выполняется проверка `isfinite` (защита от NaN/Inf). Ключевая идея в том, чтобы реже парсить выражение, и чаще вычислять его.

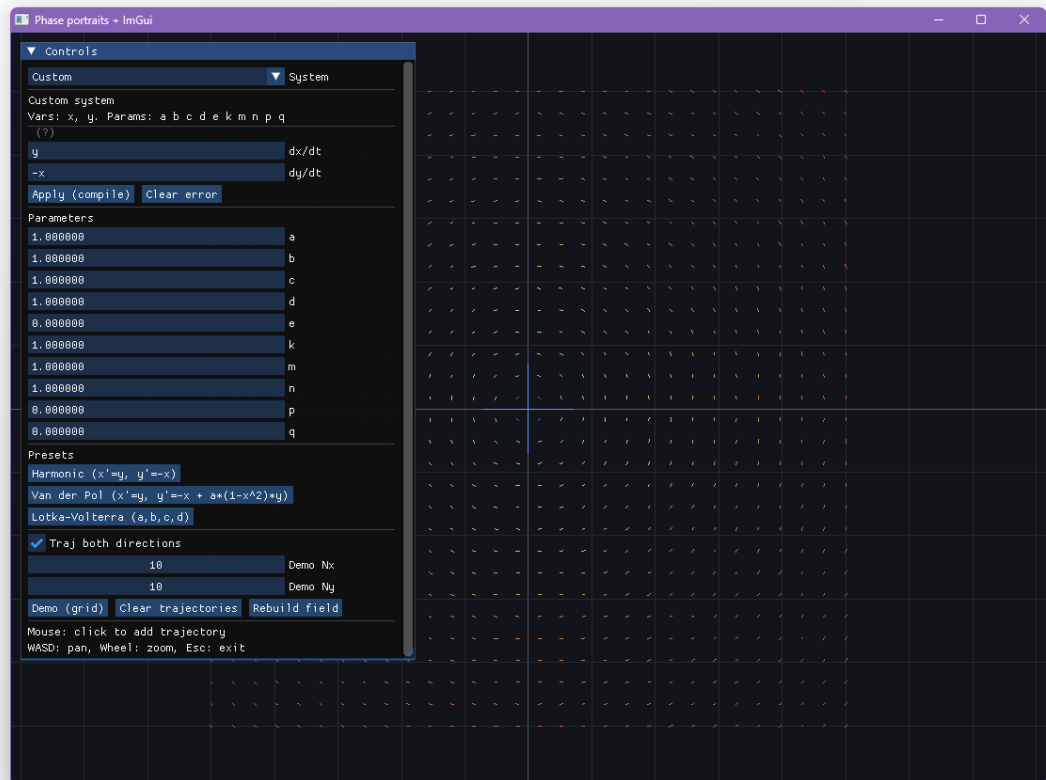


Рис. 7. Окно ввода пользовательской системы

4.5. Камера, координаты мира и управление

Файл `camera2d.h` реализует 2D-камеру для ортографической визуализации. Камера хранит:

- *center* – центр видимой области в мировых координатах;
- *worldHeight* – высота видимого мира (ширина определяется соотношением сторон окна);
- *viewportW*, *viewportH* – размер окна.

Метод `VP()` строит ортографическую матрицу `glm::ortho(l, r, b, t, -1, 1)`, которая переводит мировые координаты в clip-space OpenGL. Матрица передаётся в шейдер как *uniform* `uVP`.

Метод `screenToWorld(sx, sy)` преобразует координаты курсора (в пикселях) в мировые:

- нормирует `sx, sy` в диапазон $[-1, 1][[-1, 1]$,
- масштабирует на *half-extents* и сдвигает на *center*.

Это используется при клике мышью для задания начальной точки траектории; *zoom(factor)* масштабирует *worldHeight* с защитой от слишком маленьких или больших значений; *pan(deltaWorld)* двигает *center*. В *kursovaya.cpp* пан реализован клавишами WASD, зум — колесом мыши.

4.6. Подсистема визуализации

Используется структура:

```
struct VertexPC {  
    glm::vec2 pos;  
    glm::vec3 color;  
};
```

В ней на каждую вершину задаются позиция и цвет.

Файл *renderer2d.h* содержит реализацию двух типов батчей.

LinesBatch нужен для сетки, поля направлений, равновесий, и хранит CPU-массив вершин *cpuVertices*, OpenGL объекты *vao*, *vbo*, а также методы:

- *initGL()* – создаёт VAO/VBO и описывает layout атрибутов - *location 0: vec2 pos, location 1: vec3 color*;
- *addSegment(a,b,color)* – добавляет 2 вершины в массив;
- *upload()* – *glBufferData*, отправляет CPU-массив в VBO;
- *draw()* – *glDrawArrays(GL_LINES, ...)*.

PolylineBatch предназначен для отрисовки множества независимых траекторий:

- общий массив вершин *cpuVertices*,
- список диапазонов *ranges*, где каждый диапазон соответствует отдельной полилинии (*first, count*).

Методы:

- *addPolylineVertices(verts)* – добавляет вершины и сохраняет диапазон;
- *upload()* – загружает весь массив;

- *draw()* – в цикле по *ranges* вызывает *glDrawArrays(GL_LINE_STRIP, first, count)*.

Это позволяет хранить и рисовать множество траекторий эффективно, без отдельного VBO на каждую.

4.7. Шейдеры и создание программы OpenGL

Здесь рассматривается два файла – *shaders.h* и *gl_utils.h*. В программе используется простая шейдерная пара из вершинного и фрагментного шейдеров. Vertex shader принимает *aPos*, *aColor*, умножает позицию на *uVP*, передаёт цвет дальше: *gl_Position = uVP * vec4(aPos, 0.0, 1.0)*. Fragment shader выводит цвет: *FragColor = vec4(vColor, 1.0)*;

Такая схема подходит для линий и полилиний и делает цветовую визуализацию (скорость, тип равновесий) простой.

В *gl_utils.h* есть функции:

- *compileShader(type, src)* – компилирует шейдер и печатает лог ошибок;
- *linkProgram(vs, fs)* – линкует программу, печатает лог ошибок, удаляет шейдеры после линковки.

В *kursovaaya.cpp* это используется при старте:

```
GLuint vs = compileShader(GL_VERTEX_SHADER, kLineVS);
GLuint fs = compileShader(GL_FRAGMENT_SHADER, kLineFS);
st.program = linkProgram(vs, fs);
st.uVP = glGetUniformLocation(st.program, "uVP");
```

4.8. Главный файл приложения и цикл работы

kursovaaya.cpp связывает все модули в одно приложение: создаёт окно, инициализирует OpenGL и ImGui, формирует геометрию и рисует каждый кадр. Рассмотрим файл по порядку. Сначала нужно построить геометрию сцены с помощью *rebuild*-функций. Всего их три – *rebuildGridAxes*, *rebuildVectorField* и *rebuildEquilibria*.

1. Сетка и оси: *rebuildGridAxes*

- подбирается шаг сетки *step* по масштабу камеры;
- вычисляются границы видимой области;
- строятся вертикальные и горизонтальные линии;
- оси ($x=0$, $y=0$) рисуются другим цветом;
- данные загружаются в *gridAxes* (*upload*).

2. Поле направлений: *rebuildVectorField*

- вызывается *sampleVectorField(sys, worldBox, nx, ny)*;
- для каждого *sample* вычисляется скорость $|v|$, нормируется направление;
- строится короткий сегмент фиксированной длины;
- цвет сегмента – “тепловая шкала” *heatColor(t)* в зависимости от скорости;
- сегменты добавляются в *vectorField* и загружаются.

3. Равновесия: *rebuildEquilibria*

- вызывается *findEquilibriaGridNewton(...)*;
- каждая точка анализируется *analyzeEquilibrium*;
- цвет равновесия задаётся функцией *eqColor(type)*;
- рисуются крестики (2 сегмента на точку) в *equilibriaCrosses*.

Траектория добавляется функцией *addTrajectoryFromSeed(st, seedWorld, bothDirections)*. Внутри функции выполняется интегрирование (*integrateTrajectory* из *phase_math.h*), рисование (*PolylineBatch*). Цвет вершины зависит от скорости в точке (так траектория “подсвечивает” ускорение движения). Если включён режим “в обе стороны”, строится *forward* и *backward*, *backward* затемняется. Функция *addSeedGrid* генерирует сетку начальных условий и многократно вызывает добавление траектории.

В *main()* создается окно *GLFW*, загружается *GLAD*, инициализируется *ImGui* (*InitForOpenGL(window, false)*) и задаются *callbacks* для мыши и клавиатуры. Важно отметить, что в *callbacks* сначала вызываются функции

backend ImGui, вроде `ImGui_ImplGlfw_*Callback`. Логика программы выполняется только если `!ImGui::GetIO().WantCaptureMouse` (для кликов/скролла) и `!ImGui::GetIO().WantCaptureKeyboard` (для клавиатуры). Это нужно, чтобы UI корректно принимал ввод текста и клики не “пробивали” интерфейс.

Далее главный цикл отрисовки. Для каждого кадра обновляются размеры viewport и применяется панорамирование WASD, если ImGui не захватил клавиатуру. В создаваемом окне есть выбор системы, пресеты, кастомный ввод с параметрами, а также демонстративная сетка. Возможно очищение траекторий и пересборка поля (рис.8).

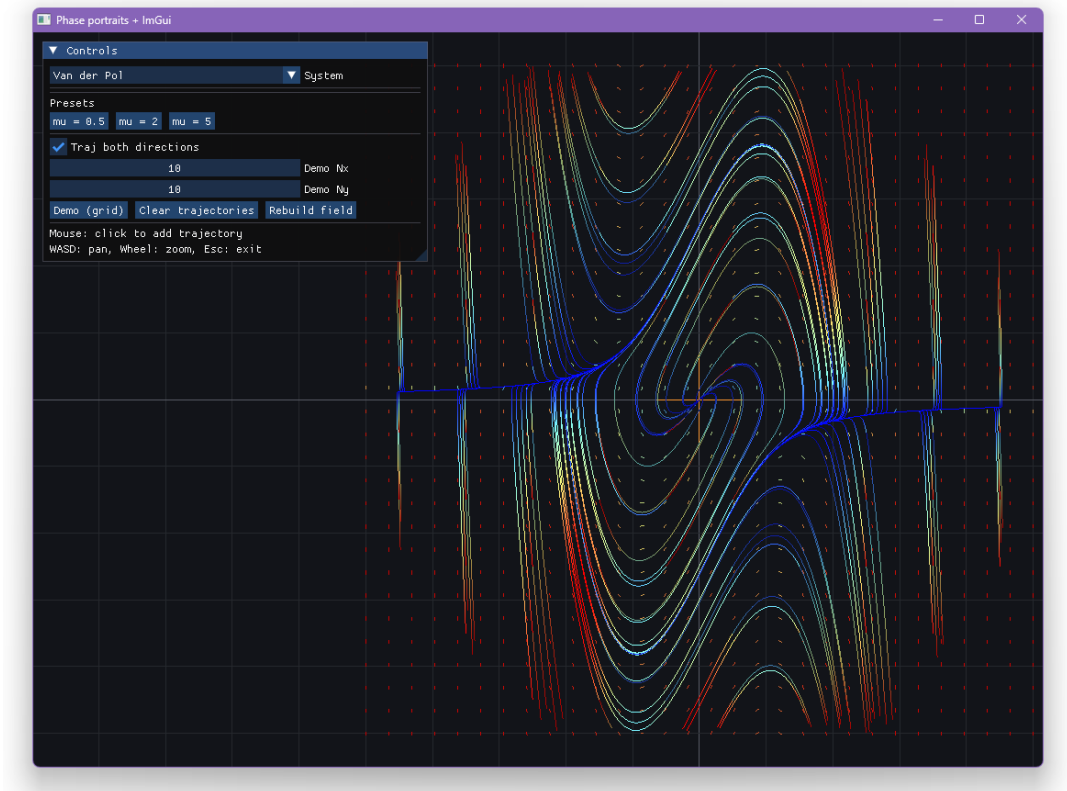


Рис. 8. Как выглядит окно на примере пресета Ван-дер-Поль

5. Вычислительный эксперимент и демонстрация работы

5.1. Линейные системы

Для проверки визуализации и корректности интегрирования были рассмотрены базовые линейные системы.

1. Центр. $\dot{x} = y, \dot{y} = -x$. Ожидаемое поведение – семейство замкнутых орбит вокруг начала координат (рис. 10).

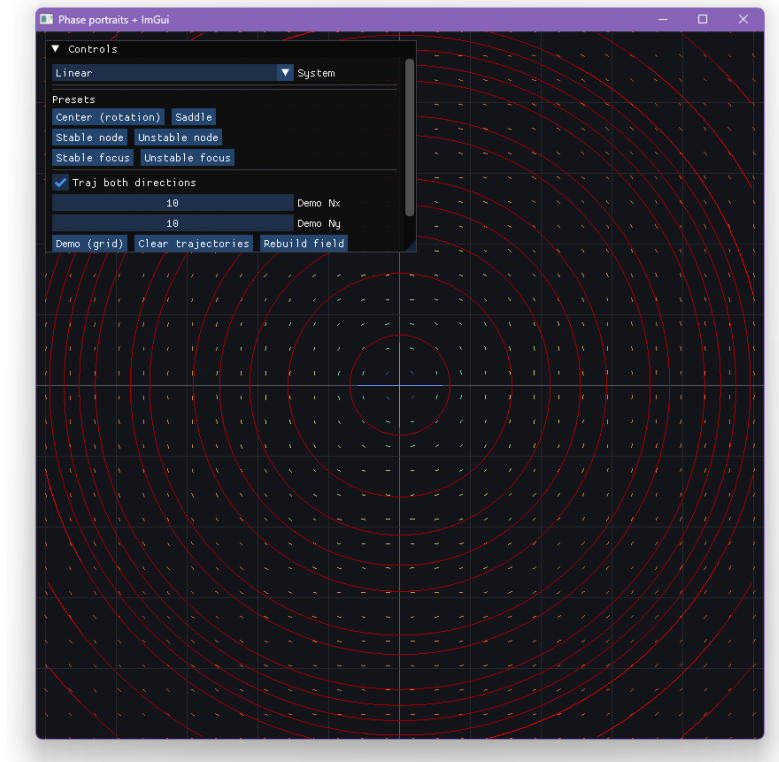


Рис. 10. Гармонический осциллятор: замкнутые линии

2. Седло. $\dot{x} = x, \dot{y} = -y$. Ожидаемое поведение – устойчивые и неустойчивые сепаратрисы в равных количествах (рис. 11).

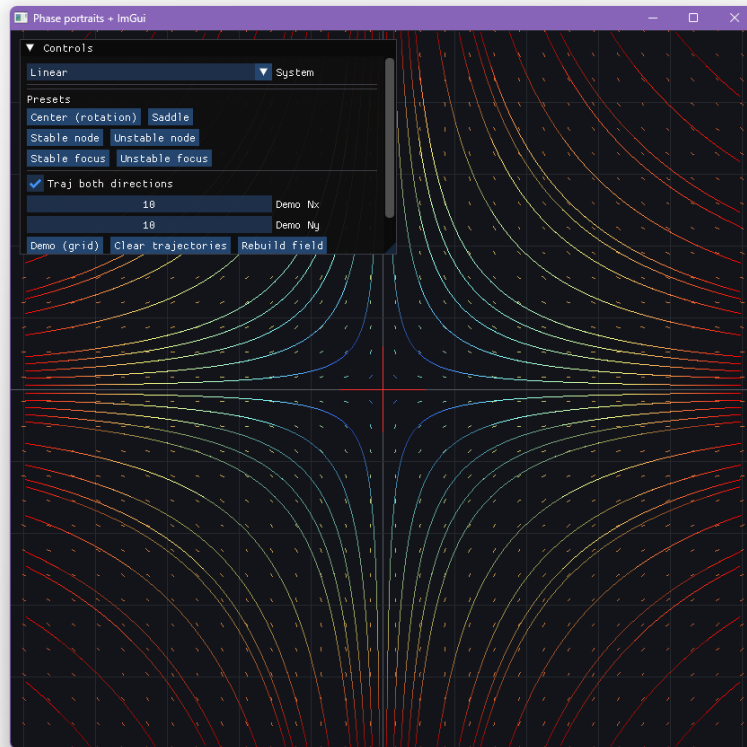


Рис. 11. Седло: сепаратрисы и характер траекторий

3. Устойчивый фокус. $\dot{x} = ax - y, \dot{y} = x + ay, a < 0$. При $a = -0,2$ траектории закручиваются и стремятся к равновесию (рис. 12).

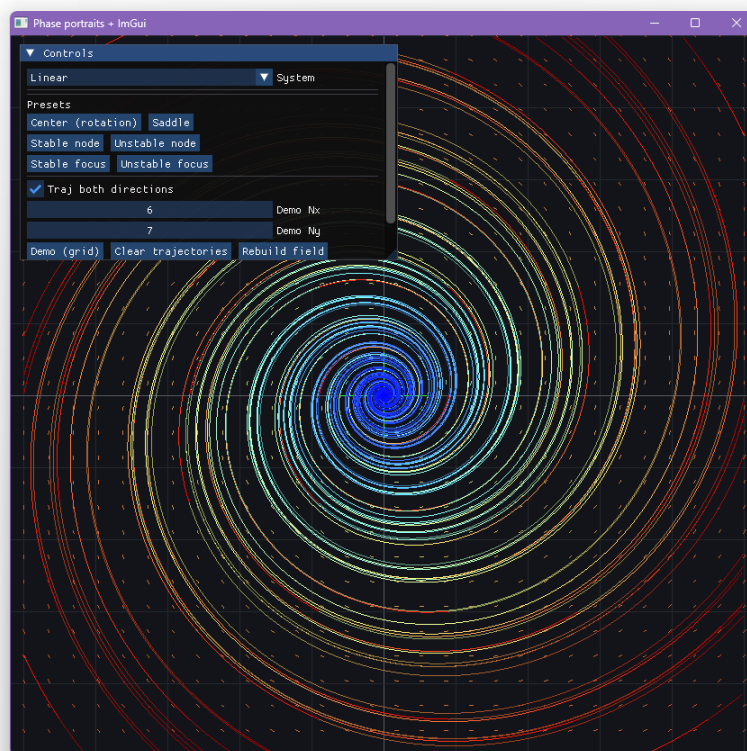


Рис. 12. Устойчивый фокус: траектории-спирали

5.2. Осциллятор Ван-дер-Поля

Система Ван-дер-Поля имеет вид:

$$\dot{x} = y, \dot{y} = -x + a(1 - x^2)y.$$

Это математическая модель, которая описывает колебания в различных колебательных системах, например, в автогенераторах на электронных лампах, биполярных и полевых транзисторах. При $a > 0$ наблюдается формирование устойчивого предельного цикла: траектории из различных начальных условий стремятся к одной замкнутой кривой (рис. 13).

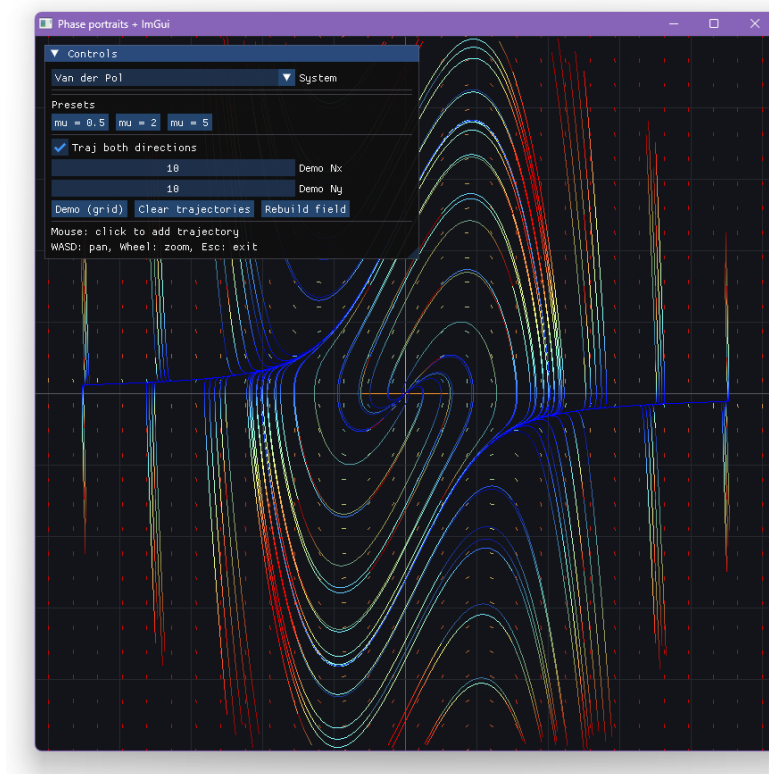


Рис. 13. Ван-дер-Поль: предельный цикл

4.3. Модель Лотки-Вольтерра

Представляет собой модель «хищник-жертва»:

$$\dot{x} = ax - bxy, \dot{y} = -cy + dxy.$$

Хотя уравнение не учитывает, например, ограниченность ресурсов для жертв, возрастные структуры популяций или внешние факторы, модель Лотки-Вольтерры демонстрирует, как простое взаимодействие двух видов

может приводить к сложным динамическим эффектам: регулярным колебаниям численности. В отношении фазовых портретов, при типичных параметрах система демонстрирует замкнутые траектории положительного равновесия (рис. 14).

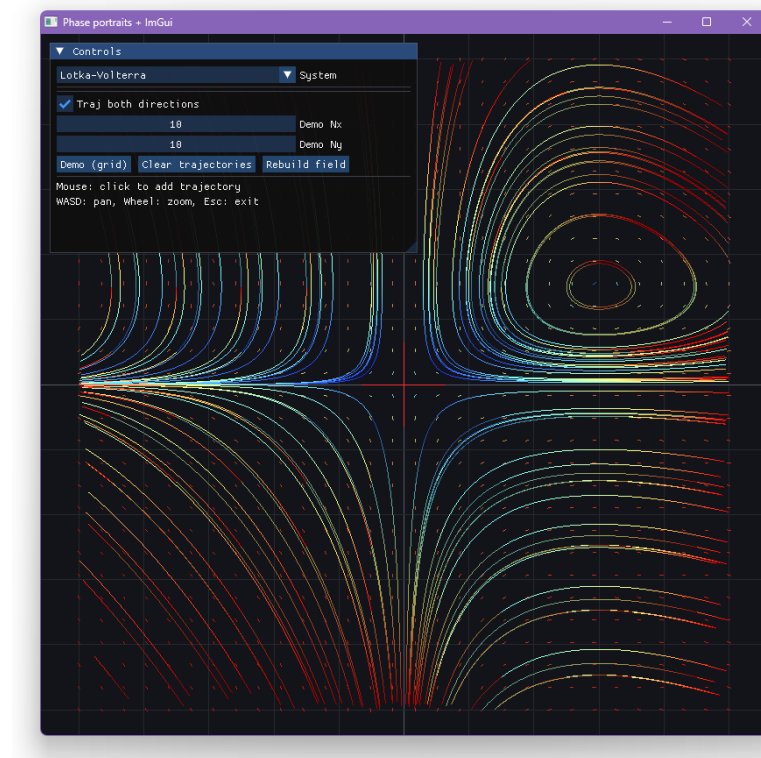


Рис. 14. Лотка-Вольтерра: семейство замкнутых траекторий

4.4. Пользовательский ввод

В Custom-режиме были протестированы:

- корректный ввод формул (включая степенные функции и тригонометрию);
- обработка ошибок синтаксиса (сообщение о позиции ошибки);
- изменение параметров и корректный пересчёт поля и равновесий;
- ввод дробных значений параметров с десятичной запятой.

Отмечено, что компиляция выражения выполняется только по кнопке Apply, что обеспечивает достаточную производительность: при обычном перемещении камеры и добавлении траекторий вычисляется только *te_eval*, а не повторный парсинг строк.

В качестве демонстрации была введена нелинейная система с несколькими равновесиями и вращением (рис. 15), в следующем виде:

$$\dot{x} = y - x(x^2 + y^2 - a), \dot{y} = -x - y(x^2 + y^2 - a), a = 1$$

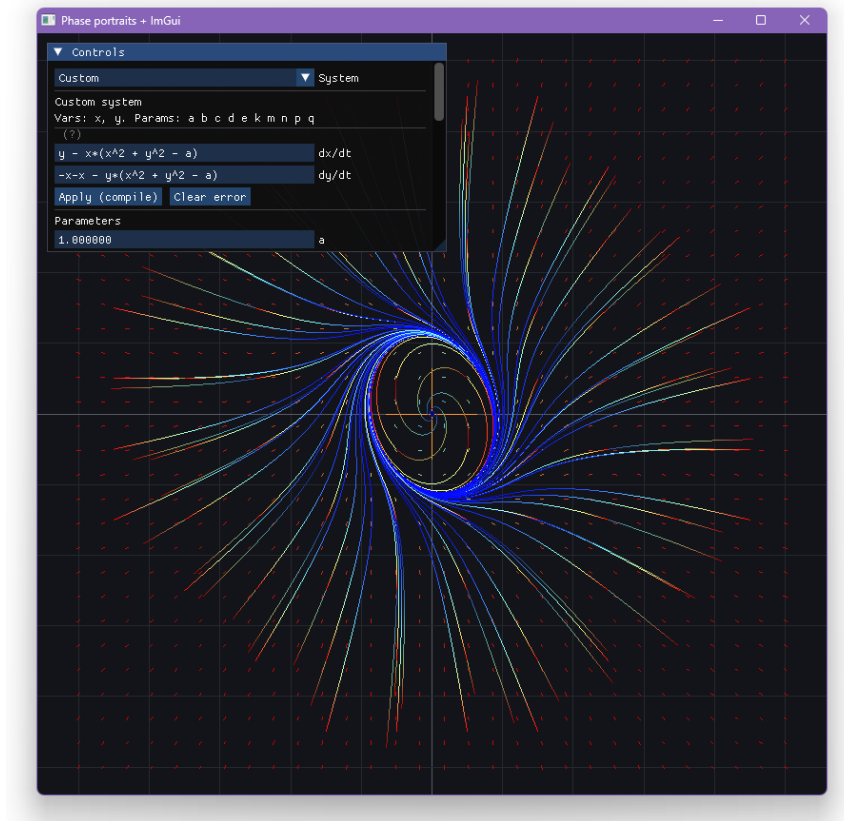


Рис. 15. Проверка Custom-режима

Система демонстрирует автоколебательное поведение с устойчивым предельным циклом. Независимо от начального положения (кроме точки равновесия), система приходит к регулярным колебаниям на окружности единичного радиуса, или близких к ней формам. Такая система может применяться, например, в вышеупомянутых колебаниях Ван-дер-Поля, в описании биоритмов организма или в финансовых моделях рынков с саморегулированием.

Заключение

В результате выполнения курсовой работы разработано программное приложение для интерактивной визуализации фазовых портретов двумерных автономных систем дифференциальных уравнений. Были реализованы:

- построение поля направлений на сетке;
- численное интегрирование траекторий методом Рунге–Кутты 4-го порядка;
- поиск и отображение точек равновесия и их классификация по Якобиану;
- пользовательский ввод правых частей системы и параметров с использованием `tinyexpr`;
- графическая визуализация на OpenGL и интерфейс управления на ImGui.

Работоспособность программы подтверждена вычислительным экспериментом на наборе типовых систем: линейные системы (центр, седло, фокус), осциллятор Ван-дер-Поля, модель Лотки–Вольтерры, а также произвольная пользовательская система.

В перспективе развития есть добавление адаптивного шага интегрирования, улучшение алгоритма поиска равновесий, экспорт траекторий и/или изображений, расширение набора встроенных систем и режимов визуализации.

Список использованных источников

1. Hairer E., Nørsett S. P., Wanner G. Solving Ordinary Differential Equations I: Nonstiff Problems. – Berlin : Springer, 1993. – 528 p.
2. Perko L. Differential Equations and Dynamical Systems. – New York : Springer, 2001. – 553 p.
3. Задорожний, В.Г. Дифференциальные уравнения с вариационными производными // В.Г. Задорожний; М-во образования Рос. Федерации. Воронеж. гос. ун-т. — Воронеж, 2000. — 368 с.
4. Andronov A. A., Vitt A. A., Khaikin S. E. Theory of Oscillators. – Oxford : Pergamon Press, 1966. – 815 p.
5. Press W. H., Teukolsky S. A., Vetterling W. T., Flannery B. P. Numerical Recipes: The Art of Scientific Computing. – Cambridge : Cambridge University Press, 2007. – 1235 p.
6. OpenGL Registry (спецификации OpenGL). – URL: <https://www.khronos.org/opengl/> (дата обращения: 23.03.2026).
7. GLFW Documentation. – URL: <https://www.glfw.org/documentation.html> (дата обращения: 12.04.2026).
8. Dear ImGui (официальный репозиторий и документация). – URL: <https://github.com/ocornut/imgui> (дата обращения: 15.04.2026).
9. tinyexpr: Tiny recursive-descent expression parser. – URL: <https://github.com/codeplea/tinyexpr> (дата обращения: 15.04.2026).

Приложение

```
kursovaya.cpp
#include <glad/glad.h>
#include <GLFW/glfw3.h>

#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
#include <cstring>

#include "phase_math.h"
#include "systems.h"
#include "custom_system.h"
#include "camera2d.h"
#include "gl_utils.h"
#include "renderer2d.h"
#include "shaders.h"

#include "imgui.h"
#include "backends/imgui_impl_glfw.h"
#include "backends/imgui_impl_opengl3.h"

static void framebuffer_size_callback(GLFWwindow*, int width, int height) {
    glViewport(0, 0, width, height);
}

static float clamp01(float x) { return std::max(0.0f, std::min(1.0f, x)); }
static glm::vec3 heatColor(float t) {
    t = clamp01(t);
    float r = clamp01(1.5f * t);
    float g = clamp01(1.5f * (1.0f - std::abs(t - 0.5f) * 2.0f));
    float b = clamp01(1.5f * (1.0f - t));
    return { r, g, b };
}

enum class SystemKind { Linear = 0, VanDerPol = 1, Lotka = 2, Custom = 3 };

struct AppState {
    Camera2D cam;
    phase::AABB worldBox{ {-5,-5},{5,5} };

    phase::LinearSystem linear;
    phase::VanDerPol vdp;
    phase::LotkaVolterra lotka;
    phase::CustomSystem custom; // NEW

    SystemKind current = SystemKind::VanDerPol;

    const phase::ISystem2D& sys() const {
        switch (current) {
            case SystemKind::Linear:    return linear;
            case SystemKind::VanDerPol: return vdp;
            case SystemKind::Lotka:     return lotka;
            case SystemKind::Custom:    return custom;
        }
        return vdp;
    }
}
```

```

phase::TrajectorySettings trajS;

PolylineBatch trajectories;
LinesBatch vectorField;
LinesBatch gridAxes;
LinesBatch equilibriaCrosses;

GLuint program = 0;
GLint uVP = -1;

std::vector<phase::EquilibriumInfo> eqInfos;

// GUI state
int demoNx = 10;
int demoNy = 10;
bool bothDirections = true;

char fxBuf[256] = "y";
char fyBuf[256] = "-x";
std::string customError;
bool autoApplyCustom = false;
};

static void rebuildGridAxes(AppState& st) {
    st.gridAxes.clear();

    double h = st.cam.worldHeight;
    double step = 1.0;
    if (h > 40) step = 5.0;
    if (h > 120) step = 10.0;
    if (h < 10) step = 0.5;
    if (h < 4) step = 0.2;

    auto he = st.cam.halfExtents();
    double l = st.cam.center.x - he.x;
    double r = st.cam.center.x + he.x;
    double b = st.cam.center.y - he.y;
    double t = st.cam.center.y + he.y;

    l -= step; r += step; b -= step; t += step;

    auto gridCol = glm::vec3(0.15f, 0.16f, 0.18f);
    auto axisCol = glm::vec3(0.35f, 0.36f, 0.40f);

    auto snapDown = [&](double x) { return std::floor(x / step) * step; };
    auto snapUp = [&](double x) { return std::ceil(x / step) * step; };

    for (double x = snapDown(l); x <= snapUp(r); x += step) {
        glm::vec3 col = (std::abs(x) < 1e-12) ? axisCol : gridCol;
        st.gridAxes.addSegment(glm::vec2((float)x, (float)b), glm::vec2((float)x,
(float)t), col);
    }
    for (double y = snapDown(b); y <= snapUp(t); y += step) {
        glm::vec3 col = (std::abs(y) < 1e-12) ? axisCol : gridCol;
        st.gridAxes.addSegment(glm::vec2((float)l, (float)y), glm::vec2((float)r,
(float)y), col);
    }

    st.gridAxes.upload(GL_DYNAMIC_DRAW);
}

```



```

// ----- vector field -----
static void rebuildVectorField(AppState& st, int nx = 30, int ny = 30) {
    st.vectorField.clear();
    auto samples = phase::sampleVectorField(st.sys(), st.worldBox, nx, ny);

    double len = 0.18 * (st.worldBox.max.y - st.worldBox.min.y) / double(ny);

    for (auto& s : samples) {
        double speed = glm::length(s.v);
        if (!std::isfinite(speed) || speed < 1e-12) continue;

        glm::dvec2 dir = s.v / speed;
        glm::dvec2 a = s.p - 0.5 * len * dir;
        glm::dvec2 b = s.p + 0.5 * len * dir;

        float t = (float)(speed / (1.0 + speed));
        glm::vec3 col = heatColor(t) * 0.8f;

        st.vectorField.addSegment(glm::vec2(a), glm::vec2(b), col);
    }
    st.vectorField.upload(GL_DYNAMIC_DRAW);
}

static glm::vec3 eqColor(phase::EquilibriumType tp) {
    using T = phase::EquilibriumType;
    switch (tp) {
        case T::Saddle:      return { 1.0f, 0.2f, 0.2f };
        case T::StableNode:  return { 0.2f, 1.0f, 0.2f };
        case T::StableFocus: return { 0.2f, 1.0f, 0.2f };
        case T::UnstableNode: return { 1.0f, 0.6f, 0.2f };
        case T::UnstableFocus: return { 1.0f, 0.6f, 0.2f };
        case T::Center:      return { 0.4f, 0.6f, 1.0f };
        default:             return { 0.9f, 0.9f, 0.9f };
    }
}

static void rebuildEquilibria(AppState& st) {
    st.eqInfos.clear();
    st.equilibriaCrosses.clear();

    auto roots = phase::findEquilibriaGridNewton(st.sys(), st.worldBox,
        45, 45,
        1e-2,
        1e-3);

    double cross = 0.06 * st.cam.worldHeight;
    for (auto& r : roots) {
        auto info = phase::analyzeEquilibrium(st.sys(), r);
        st.eqInfos.push_back(info);

        glm::vec3 col = eqColor(info.type);
        glm::vec2 p = glm::vec2((float)r.x, (float)r.y);

        st.equilibriaCrosses.addSegment(p + glm::vec2(-(float)cross, 0), p +
glm::vec2((float)cross, 0), col);
        st.equilibriaCrosses.addSegment(p + glm::vec2(0, -(float)cross), p +
glm::vec2(0, (float)cross), col);
    }

    st.equilibriaCrosses.upload(GL_DYNAMIC_DRAW);
}

```

```

static void rebuildAll(AppState& st) {
    rebuildVectorField(st);
    rebuildEquilibria(st);
    rebuildGridAxes(st);
}

static void clearTrajectories(AppState& st) {
    st.trajectories.clear();
    st.trajectories.upload(GL_DYNAMIC_DRAW);
}

static void addTrajectoryFromSeed(AppState& st, glm::dvec2 seedWorld, bool
bothDirections = true) {
    st.trajS.bounds = st.worldBox;

    auto buildVertices = [&](const std::vector<phase::Vec2>& pts, bool backward) {
        std::vector<VertexPC> v;
        v.reserve(pts.size());

        double vmax = 0.0;
        for (auto& p : pts) vmax = std::max(vmax, (double)glm::length(st.sys().F(p)));
        if (vmax < 1e-12) vmax = 1.0;

        for (auto& p : pts) {
            double speed = glm::length(st.sys().F(p));
            float t = (float)(speed / vmax);
            glm::vec3 col = heatColor(t);
            if (backward) col *= 0.65f;
            v.push_back({ glm::vec2((float)p.x, (float)p.y), col });
        }
        return v;
    };

    st.trajS.dir = phase::TimeDirection::Forward;
    auto trF = phase::integrateTrajectory(st.sys(), phase::Vec2(seedWorld), st.trajS);
    if (trF.points.size() >= 2)
        st.trajectories.addPolylineVertices(buildVertices(trF.points, false));

    if (bothDirections) {
        st.trajS.dir = phase::TimeDirection::Backward;
        auto trB = phase::integrateTrajectory(st.sys(), phase::Vec2(seedWorld),
st.trajS);
        if (trB.points.size() >= 2)
            st.trajectories.addPolylineVertices(buildVertices(trB.points, true));
    }

    st.trajectories.upload(GL_DYNAMIC_DRAW);
}

static void addSeedGrid(AppState& st, int nx = 10, int ny = 10, bool both = true) {
    double mx = 0.05 * (st.worldBox.max.x - st.worldBox.min.x);
    double my = 0.05 * (st.worldBox.max.y - st.worldBox.min.y);

    for (int j = 0; j < ny; ++j) {
        double ty = (ny == 1) ? 0.5 : double(j) / double(ny - 1);
        double y = (st.worldBox.min.y + my) + ty * ((st.worldBox.max.y - my) -
(st.worldBox.min.y + my));

        for (int i = 0; i < nx; ++i) {
            double tx = (nx == 1) ? 0.5 : double(i) / double(nx - 1);

```

```

        double x = (st.worldBox.min.x + mx) + tx * ((st.worldBox.max.x - mx) -
(st.worldBox.min.x + mx));
        addTrajectoryFromSeed(st, { x,y }, both);
    }
}

int main() {
    glfwInit();
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

    GLFWwindow* window = glfwCreateWindow(1100, 800, "Phase portraits + ImGui", nullptr,
nullptr);
    if (!window) { glfwTerminate(); return -1; }
    glfwMakeContextCurrent(window);
    glfwSetFramebufferSizeCallback(window, framebuffer_size_callback);

    if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress)) {
        std::cerr << "Failed to init GLAD\n";
        return -1;
    }

    IMGUI_CHECKVERSION();
    ImGui::CreateContext();
    ImGui::StyleColorsDark();
    ImGui_ImplGlfw_InitForOpenGL(window, false);
    ImGui_ImplOpenGL3_Init("#version 330");

    AppState st;
    glfwGetFramebufferSize(window, &st.cam.viewportW, &st.cam.viewportH);
    st.cam.center = { 0,0 };
    st.cam.worldHeight = 10.0;

    st.linear.a = 0; st.linear.b = 1; st.linear.c = -1; st.linear.d = 0; // rotation
    st.vdp.mu = 2.0;

    st.trajS.h = 0.01;
    st.trajS.maxSteps = 15000;
    st.trajS.maxDelta = 0.8;
    st.trajS.stopSpeedEps = 1e-6;

    GLuint vs = compileShader(GL_VERTEX_SHADER, kLineVS);
    GLuint fs = compileShader(GL_FRAGMENT_SHADER, kLineFS);
    st.program = linkProgram(vs, fs);
    st.uVP = glGetUniformLocation(st.program, "uVP");

    st.trajectories.initGL();
    st.vectorField.initGL();
    st.gridAxes.initGL();
    st.equilibriaCrosses.initGL();

    strncpy_s(st.fxBuf, sizeof(st.fxBuf), st.custom.fxStr.c_str(), _TRUNCATE);
    strncpy_s(st.fyBuf, sizeof(st.fyBuf), st.custom.fyStr.c_str(), _TRUNCATE);
    st.fxBuf[sizeof(st.fxBuf) - 1] = 0;
    st.fyBuf[sizeof(st.fyBuf) - 1] = 0;

    rebuildAll(st);

    glfwSetWindowUserPointer(window, &st);

```

```

    glfwSetMouseButtonCallback(window, [](GLFWwindow* w, int button, int action, int
mods)
    {
        ImGui_ImplGlfw_MouseButtonCallback(w, button, action, mods);

        if (button != GLFW_MOUSE_BUTTON_LEFT || action != GLFW_PRESS) return;
        if (ImGui::GetIO().WantCaptureMouse) return;

        auto* st = (AppState*)glfwGetWindowUserPointer(w);
        double mx, my;
        glfwGetCursorPos(w, &mx, &my);
        glm::dvec2 seed = st->cam.screenToWorld(mx, my);
        addTrajectoryFromSeed(*st, seed, st->bothDirections);
    });

    glfwSetScrollCallback(window, [](GLFWwindow* w, double xoff, double yoff)
    {
        ImGui_ImplGlfw_ScrollCallback(w, xoff, yoff);
        if (ImGui::GetIO().WantCaptureMouse) return;

        auto* st = (AppState*)glfwGetWindowUserPointer(w);
        if (yoff > 0) st->cam.zoom(0.9);
        if (yoff < 0) st->cam.zoom(1.1);
        rebuildGridAxes(*st);
        rebuildEquilibria(*st);
    });

    glfwSetKeyCallback(window, [](GLFWwindow* w, int key, int scancode, int action,
int mods)
    {
        ImGui_ImplGlfw_KeyCallback(w, key, scancode, action, mods);

        if (ImGui::GetIO().WantCaptureKeyboard)
            return;

        if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS)
            glfwSetWindowShouldClose(w, true);
    });

    glfwSetCharCallback(window, [](GLFWwindow* w, unsigned int c)
    {
        ImGui_ImplGlfw_CharCallback(w, c);
    });

    glfwSetCursorPosCallback(window, [](GLFWwindow* w, double x, double y)
    {
        ImGui_ImplGlfw_CursorPosCallback(w, x, y);
    });

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

    while (!glfwWindowShouldClose(window)) {
        glfwGetFramebufferSize(window, &st.cam.viewportW, &st.cam.viewportH);

        double panStep = 0.02 * st.cam.worldHeight;
        if (!ImGui::GetIO().WantCaptureKeyboard) {
            if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS) st.cam.pan({ -panStep,
0 });

```

```

        if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS) st.cam.pan({ panStep,
0 });
        if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS) st.cam.pan({ 0,
panStep });
        if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS) st.cam.pan({ 0, -
panStep });
    }

    ImGui_ImplOpenGL3_NewFrame();
    ImGui_ImplGlfw_NewFrame();
    ImGui::NewFrame();

    ImGui::SetNextWindowPos(ImVec2(10, 10), ImGuiCond_Once);
    ImGui::SetNextWindowSize(ImVec2(420, 0), ImGuiCond_Once);
    ImGui::Begin("Controls");

    const char* items[] = { "Linear", "Van der Pol", "Lotka-Volterra", "Custom" };
    int cur = (int)st.current;
    if (ImGui::Combo("System", &cur, items, IM_ARRAYSIZE(items))) {
        st.current = (SystemKind)cur;
        clearTrajectories(st);
        rebuildAll(st);
    }

    ImGui::Separator();

    if (st.current == SystemKind::VanDerPol) {
        ImGui::Separator();
        ImGui::Text("Presets");

        if (ImGui::Button("mu = 0.5")) { st.vdp.mu = 0.5; clearTrajectories(st);
rebuildAll(st); }
        ImGui::SameLine();
        if (ImGui::Button("mu = 2")) { st.vdp.mu = 2.0; clearTrajectories(st);
rebuildAll(st); }
        ImGui::SameLine();
        if (ImGui::Button("mu = 5")) { st.vdp.mu = 5.0; clearTrajectories(st);
rebuildAll(st); }
    }

    if (st.current == SystemKind::Linear) {
        ImGui::Separator();
        ImGui::Text("Presets");

        auto applyLinear = [&](double a, double b, double c, double d) {
            st.linear.a = a; st.linear.b = b; st.linear.c = c; st.linear.d = d;
            clearTrajectories(st);
            rebuildAll(st);
        };

        if (ImGui::Button("Center (rotation)")) {
            applyLinear(0, 1, -1, 0);
        }
        ImGui::SameLine();
        if (ImGui::Button("Saddle")) {
            applyLinear(1, 0, 0, -1);
        }

        if (ImGui::Button("Stable node")) {
            applyLinear(-1, 0, 0, -0.4);
        }
    }

```

```

    ImGui::SameLine();
    if (ImGui::Button("Unstable node")) {
        applyLinear(1, 0, 0, 0.5);
    }

    if (ImGui::Button("Stable focus")) {
        applyLinear(-0.2, 1.0, -1.0, -0.2);
    }
    ImGui::SameLine();
    if (ImGui::Button("Unstable focus")) {
        applyLinear(0.2, 1.0, -1.0, 0.2);
    }
}

if (st.current == SystemKind::Custom)
{
    ImGui::TextUnformatted("Custom system");
    ImGui::TextUnformatted("Vars: x, y. Params: a b c d e k m n p q");
    ImGui::Separator();

    ImGui::SameLine();
    ImGui::TextDisabled("(?)");
    if (ImGui::IsItemHovered())
    {
        ImGui::BeginTooltip();
        ImGui::TextUnformatted(
            "Operators: + - * / ^\n"
            "Functions: sin cos tan asin acos atan\n"
            "                sqrt abs exp log\n"
            "Constants: pi, e (if your tinyexpr build supports them)\n"
            "Examples:\n"
            "  dx/dt = y\n"
            "  dy/dt = -x + a*(1-x^2)*y"
        );
        ImGui::EndTooltip();
    }

    ImGui::InputText("dx/dt", st.fxBuf, IM_ARRAYSIZE(st.fxBuf));
    ImGui::InputText("dy/dt", st.fyBuf, IM_ARRAYSIZE(st.fyBuf));

    bool applyPressed = ImGui::Button("Apply (compile)");
    ImGui::SameLine();
    if (ImGui::Button("Clear error")) st.customError.clear();

    if (applyPressed)
    {
        std::string err;
        const bool ok = st.custom.compile(st.fxBuf, st.fyBuf, err);
        st.customError = ok ? "" : err;

        if (ok)
        {
            clearTrajectories(st);
            rebuildAll(st);
        }
    }

    if (!st.customError.empty())
    {
        ImGui::TextColored(ImVec4(1, 0.35f, 0.35f, 1), "Error: %s",
st.customError.c_str());
    }
}

```

```

    }

    ImGui::Separator();
    ImGui::TextUnformatted("Parameters");
    bool pChanged = false;

    // параметры через Scalar-обёртку
    pChanged |= ImGui::InputDouble("a", &st.custom.a, 0.0, 0.0, "%.6f");
    pChanged |= ImGui::InputDouble("b", &st.custom.b, 0.0, 0.0, "%.6f");
    pChanged |= ImGui::InputDouble("c", &st.custom.c, 0.0, 0.0, "%.6f");
    pChanged |= ImGui::InputDouble("d", &st.custom.d, 0.0, 0.0, "%.6f");
    pChanged |= ImGui::InputDouble("e", &st.custom.e, 0.0, 0.0, "%.6f");
    pChanged |= ImGui::InputDouble("k", &st.custom.k, 0.0, 0.0, "%.6f");
    pChanged |= ImGui::InputDouble("m", &st.custom.m, 0.0, 0.0, "%.6f");
    pChanged |= ImGui::InputDouble("n", &st.custom.n, 0.0, 0.0, "%.6f");
    pChanged |= ImGui::InputDouble("p", &st.custom.p, 0.0, 0.0, "%.6f");
    pChanged |= ImGui::InputDouble("q", &st.custom.q, 0.0, 0.0, "%.6f");

    if (pChanged)
    {
        clearTrajectories(st);
        rebuildAll(st);
    }

    ImGui::Separator();
    ImGui::TextUnformatted("Presets");

    auto setCustom = [&](const char* fx, const char* fy)
    {
        snprintf(st.fxBuf, IM_ARRAYSIZE(st.fxBuf), "%s", fx);
        snprintf(st.fyBuf, IM_ARRAYSIZE(st.fyBuf), "%s", fy);

        std::string err;
        bool ok = st.custom.compile(st.fxBuf, st.fyBuf, err);
        st.customError = ok ? "" : err;

        clearTrajectories(st);
        rebuildAll(st);
    };

    if (ImGui::Button("Harmonic (x'=y, y'=-x)"))
        setCustom("y", "-x");

    if (ImGui::Button("Van der Pol (x'=y, y'=-x + a*(1-x^2)*y)"))
    {
        st.custom.a = 2.0;
        setCustom("y", "-x + a*(1 - x^2)*y");
    }

    if (ImGui::Button("Lotka-Volterra (a,b,c,d)"))
    {
        st.custom.a = 1.5; st.custom.b = 1.0; st.custom.c = 1.0; st.custom.d
= 1.0;
        setCustom("a*x - b*x*y", "-c*y + d*x*y");
    }
}

ImGui::Separator();
ImGui::Checkbox("Traj both directions", &st.bothDirections);

ImGui::DragInt("Demo Nx", &st.demoNx, 1, 2, 40);

```

```

    ImGui::DragInt("Demo Ny", &st.demoNy, 1, 2, 40);

    if (ImGui::Button("Demo (grid)")) {
        addSeedGrid(st, st.demoNx, st.demoNy, st.bothDirections);
    }
    ImGui::SameLine();
    if (ImGui::Button("Clear trajectories")) {
        clearTrajectories(st);
    }
    ImGui::SameLine();
    if (ImGui::Button("Rebuild field")) {
        rebuildAll(st);
    }

    ImGui::Separator();
    ImGui::Text("Mouse: click to add trajectory");
    ImGui::Text("WASD: pan, Wheel: zoom, Esc: exit");

    ImGui::End();

    rebuildGridAxes(st);
    glClearColor(0.07f, 0.08f, 0.10f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glUseProgram(st.program);
    glm::mat4 VP = st.cam.VP();
    glUniformMatrix4fv(st.uVP, 1, GL_FALSE, &VP[0][0]);

    st.gridAxes.draw();
    st.vectorField.draw();
    st.equilibriaCrosses.draw();
    st.trajectories.draw();

    ImGui::Render();
    ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());

    glfwSwapBuffers(window);
    glfwPollEvents();

    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);
}

ImGui_ImplOpenGL3_Shutdown();
ImGui_ImplGlfw_Shutdown();
ImGui::DestroyContext();

st.trajectories.destroy();
st.vectorField.destroy();
st.gridAxes.destroy();
st.equilibriaCrosses.destroy();
glDeleteProgram(st.program);

glfwTerminate();
return 0;
}

```

```

camera_2d.h
#pragma once
#include <glm/glm.hpp>

```



```

#include <glm/gtc/matrix_transform.hpp>

struct Camera2D {
    glm::dvec2 center{ 0.0, 0.0 };

    double worldHeight = 10.0;

    int viewportW = 800;
    int viewportH = 600;

    double aspect() const {
        return (viewportH == 0) ? 1.0 : double(viewportW) / double(viewportH);
    }

    glm::dvec2 halfExtents() const {
        double hh = worldHeight * 0.5;
        double hw = hh * aspect();
        return { hw, hh };
    }

    glm::mat4 VP() const {
        auto he = halfExtents();
        double l = center.x - he.x;
        double r = center.x + he.x;
        double b = center.y - he.y;
        double t = center.y + he.y;
        return glm::ortho(float(l), float(r), float(b), float(t), -1.0f, 1.0f);
    }

    glm::dvec2 screenToWorld(double sx, double sy) const {
        double nx = (sx / double(viewportW)) * 2.0 - 1.0;
        double ny = 1.0 - (sy / double(viewportH)) * 2.0;

        auto he = halfExtents();
        return {
            center.x + nx * he.x,
            center.y + ny * he.y
        };
    }

    void zoom(double factor) {
        worldHeight *= factor;
        if (worldHeight < 1e-3) worldHeight = 1e-3;
        if (worldHeight > 1e6) worldHeight = 1e6;
    }

    void pan(glm::dvec2 deltaWorld) {
        center += deltaWorld;
    }
};

```

```

systems.h
#pragma once
#include "phase_math.h"

```

```

namespace phase {

    struct LinearSystem : ISystem2D {
        //  $x' = a x + b y$ 
        //  $y' = c x + d y$ 
    };
}

```

```

    double a = 1, b = 0, c = 0, d = -1;

    Vec2 F(Vec2 p) const override {
        return { a * p.x + b * p.y, c * p.x + d * p.y };
    }
    const char* name() const override { return "Linear 2x2"; }
};

struct VanDerPol : ISystem2D {
    double mu = 1.0;
    Vec2 F(Vec2 p) const override {
        double x = p.x, y = p.y;
        return { y, mu * (1.0 - x * x) * y - x };
    }
    const char* name() const override { return "Van der Pol"; }
};

struct LotkaVolterra : ISystem2D {
    // x' = ax - bxy
    // y' = -cy + dxy
    double a = 1.5, b = 1.0, c = 3.0, d = 1.0;

    Vec2 F(Vec2 p) const override {
        double x = p.x, y = p.y;
        return { a * x - b * x * y, -c * y + d * x * y };
    }
    const char* name() const override { return "Lotka-Volterra"; }
};
}

custom_system.h
#pragma once
#include <string>
#include <array>
#include <cstring>
#include <cmath>

#include "phase_math.h"
#include "tinyexpr.h"

namespace phase {

    struct CustomSystem final : ISystem2D {
        std::string fxStr = "y";
        std::string fyStr = "-x";

        double a = 1, b = 1, c = 1, d = 1, e = 0, k = 1, m = 1, n = 1, p = 0, q = 0;

        mutable double xVar = 0.0;
        mutable double yVar = 0.0;

        te_expr* fx = nullptr;
        te_expr* fy = nullptr;

        std::array<te_variable, 12> vars{};

        CustomSystem() {
            vars = { {
                {"x", &xVar},

```

```

        {"y", &yVar},
        {"a", &a},
        {"b", &b},
        {"c", &c},
        {"d", &d},
        {"e", &e},
        {"k", &k},
        {"m", &m},
        {"n", &n},
        {"p", &p},
        {"q", &q},
    } };
    std::string err;
    compile(fxStr, fyStr, err);
}

~CustomSystem() override {
    clearCompiled();
}

void clearCompiled() {
    if (fx) { te_free(fx); fx = nullptr; }
    if (fy) { te_free(fy); fy = nullptr; }
}

bool compile(const std::string& newFx, const std::string& newFy, std::string&
outError) {
    outError.clear();
    fxStr = newFx;
    fyStr = newFy;

    clearCompiled();

    int errPos = 0;
    fx = te_compile(fxStr.c_str(), vars.data(), (int)vars.size(), &errPos);
    if (!fx) {
        outError = "dx/dt parse error at position " + std::to_string(errPos);
        return false;
    }

    errPos = 0;
    fy = te_compile(fyStr.c_str(), vars.data(), (int)vars.size(), &errPos);
    if (!fy) {
        te_free(fx); fx = nullptr;
        outError = "dy/dt parse error at position " + std::to_string(errPos);
        return false;
    }

    return true;
}

Vec2 F(Vec2 p) const override {
    xVar = p.x;
    yVar = p.y;

    if (!fx || !fy) return { 0.0, 0.0 };

    double dx = te_eval(fx);
    double dy = te_eval(fy);

    if (!std::isfinite(dx) || !std::isfinite(dy)) return { 0.0, 0.0 };
}

```

```

        return { dx, dy };
    }

    const char* name() const override { return "Custom"; }
};

}

phase_math.h
#pragma once
#include <glm/glm.hpp>
#include <vector>
#include <functional>
#include <optional>
#include <cmath>
#include <cstdint>
#include <limits>

namespace phase {
    using Vec2 = glm::dvec2;
    using Mat2 = glm::dmat2;

    struct AABB {
        Vec2 min{ -10.0, -10.0 };
        Vec2 max{ 10.0, 10.0 };

        bool contains(Vec2 p) const {
            return (p.x >= min.x && p.x <= max.x && p.y >= min.y && p.y <= max.y);
        }
    };

    struct ISystem2D {
        virtual ~ISystem2D() = default;
        virtual Vec2 F(Vec2 p) const = 0;
        virtual const char* name() const = 0;
    };

    struct FunctionSystem2D final : ISystem2D {
        std::function<Vec2(Vec2)> f;
        const char* sysName = "FunctionSystem2D";

        Vec2 F(Vec2 p) const override { return f(p); }
        const char* name() const override { return sysName; }
    };

    struct RK4 {
        const ISystem2D* sys = nullptr;
        double h = 0.01;

        Vec2 step(Vec2 p) const {
            //  $p_{n+1} = p_n + (h/6)(k_1 + 2k_2 + 2k_3 + k_4)$ 
            const Vec2 k1 = sys->F(p);
            const Vec2 k2 = sys->F(p + 0.5 * h * k1);
            const Vec2 k3 = sys->F(p + 0.5 * h * k2);
            const Vec2 k4 = sys->F(p + 1.0 * h * k3);
            return p + (h / 6.0) * (k1 + 2.0 * k2 + 2.0 * k3 + k4);
        }
    };

    enum class TimeDirection { Forward, Backward };

```

```

struct TrajectorySettings {
    double h = 0.01;
    std::size_t maxSteps = 5000;
    AABB bounds;
    double maxDelta = 1.0;
    double stopSpeedEps = 1e-6;
    TimeDirection dir = TimeDirection::Forward;
};

struct Trajectory {
    std::vector<Vec2> points;
};

inline Trajectory integrateTrajectory(const ISystem2D& sys, Vec2 p0, const
TrajectorySettings& s)
{
    Trajectory tr;
    tr.points.reserve(s.maxSteps + 1);

    RK4 rk;
    rk.sys = &sys;
    rk.h = (s.dir == TimeDirection::Forward) ? s.h : -s.h;

    Vec2 p = p0;
    if (!s.bounds.contains(p)) return tr;

    tr.points.push_back(p);

    for (std::size_t i = 0; i < s.maxSteps; ++i) {
        const Vec2 v = sys.F(p);
        const double speed = glm::length(v);
        if (speed < s.stopSpeedEps) break;

        const Vec2 pn = rk.step(p);

        const double delta = glm::length(pn - p);
        if (!std::isfinite(pn.x) || !std::isfinite(pn.y)) break;
        if (delta > s.maxDelta) break;
        if (!s.bounds.contains(pn)) break;

        tr.points.push_back(pn);
        p = pn;
    }

    return tr;
}

struct FieldSample {
    Vec2 p;    // точка (x,y)
    Vec2 v;    // вектор F(p)
};

inline std::vector<FieldSample> sampleVectorField(
    const ISystem2D& sys, const AABB& box, int nx, int ny)
{
    std::vector<FieldSample> out;
    out.reserve(std::size_t(nx) * std::size_t(ny));

    for (int j = 0; j < ny; ++j) {
        const double ty = (ny == 1) ? 0.0 : double(j) / double(ny - 1);

```

```

        const double y = box.min.y + ty * (box.max.y - box.min.y);

        for (int i = 0; i < nx; ++i) {
            const double tx = (nx == 1) ? 0.0 : double(i) / double(nx - 1);
            const double x = box.min.x + tx * (box.max.x - box.min.x);

            Vec2 p{ x, y };
            Vec2 v = sys.F(p);
            out.push_back({ p, v });
        }
    }
    return out;
}

inline Mat2 jacobianCentralDiff(const ISystem2D& sys, Vec2 p, double h = 1e-5)
{
    // J = [df/dx df/dy; dg/dx dg/dy]
    const Vec2 ex{ 1.0, 0.0 };
    const Vec2 ey{ 0.0, 1.0 };

    const Vec2 Fx1 = sys.F(p + h * ex);
    const Vec2 Fx0 = sys.F(p - h * ex);
    const Vec2 Fy1 = sys.F(p + h * ey);
    const Vec2 Fy0 = sys.F(p - h * ey);

    const Vec2 dFdx = (Fx1 - Fx0) / (2.0 * h);
    const Vec2 dFdy = (Fy1 - Fy0) / (2.0 * h);

    // glm матрица по столбцам: mat2(col0, col1)
    // col0 = (df/dx, dg/dx), col1 = (df/dy, dg/dy)
    return Mat2(dFdx, dFdy);
}

enum class EquilibriumType {
    Unknown,
    Saddle,
    StableNode,
    UnstableNode,
    StableFocus,
    UnstableFocus,
    Center,
    Degenerate
};

struct EquilibriumInfo {
    Vec2 p;
    Mat2 J;
    double tr = 0.0;
    double det = 0.0;
    double disc = 0.0;
    EquilibriumType type = EquilibriumType::Unknown;
};

inline EquilibriumType classifyByTrDet(double tr, double det, double disc,
    double eps = 1e-9, double centerTrEps = 1e-6)
{
    if (std::abs(det) < eps) return EquilibriumType::Degenerate;
    if (det < 0.0) return EquilibriumType::Saddle;
    if (std::abs(disc) < eps) {
        if (tr < 0.0) return EquilibriumType::StableNode;
        if (tr > 0.0) return EquilibriumType::UnstableNode;
    }
}

```

```

        return EquilibriumType::Degenerate;
    }

    if (disc > 0.0) {
        if (tr < 0.0) return EquilibriumType::StableNode;
        if (tr > 0.0) return EquilibriumType::UnstableNode;
        return EquilibriumType::Degenerate;
    }
    else {
        if (std::abs(tr) < centerTrEps) return EquilibriumType::Center;
        if (tr < 0.0) return EquilibriumType::StableFocus;
        if (tr > 0.0) return EquilibriumType::UnstableFocus;
        return EquilibriumType::Unknown;
    }
}

inline EquilibriumInfo analyzeEquilibrium(const ISystem2D& sys, Vec2 p,
    double jacH = 1e-5)
{
    EquilibriumInfo info;
    info.p = p;
    info.J = jacobianCentralDiff(sys, p, jacH);
    info.tr = info.J[0][0] + info.J[1][1]; // trace
    info.det = info.J[0][0] * info.J[1][1] - info.J[1][0] * info.J[0][1];
    info.disc = info.tr * info.tr - 4.0 * info.det;
    info.type = classifyByTrDet(info.tr, info.det, info.disc);
    return info;
}

inline bool newtonSolveEquilibrium(
    const ISystem2D& sys,
    Vec2& p,
    int iters = 30,
    double epsF = 1e-10,
    double epsStep = 1e-12,
    double jacH = 1e-6)
{
    for (int k = 0; k < iters; ++k) {
        const Vec2 F = sys.F(p);
        if (glm::length(F) < epsF) return true;

        const Mat2 J = jacobianCentralDiff(sys, p, jacH);
        const double a = J[0][0], b = J[1][0];
        const double c = J[0][1], d = J[1][1];
        const double det = a * d - b * c;
        if (std::abs(det) < 1e-14) return false;

        const Vec2 rhs = -F;
        const Vec2 dp{
            (d * rhs.x - c * rhs.y) / det,
            (-b * rhs.x + a * rhs.y) / det
        };

        p += dp;
        if (glm::length(dp) < epsStep) return true;
    }
    return false;
}

inline std::vector<Vec2> findEquilibriaGridNewton(
    const ISystem2D& sys,

```

```

    const AABB& box,
    int nx, int ny,
    double seedSpeedThresh = 1e-2,
    double mergeDist = 1e-3)
{
    std::vector<Vec2> roots;
    roots.reserve(16);

    auto tooClose = [&](Vec2 r) {
        for (auto& q : roots)
            if (glm::length(r - q) < mergeDist) return true;
        return false;
    };

    for (int j = 0; j < ny; ++j) {
        const double ty = (ny == 1) ? 0.0 : double(j) / double(ny - 1);
        const double y = box.min.y + ty * (box.max.y - box.min.y);

        for (int i = 0; i < nx; ++i) {
            const double tx = (nx == 1) ? 0.0 : double(i) / double(nx - 1);
            const double x = box.min.x + tx * (box.max.x - box.min.x);

            Vec2 p{ x,y };
            const double speed = glm::length(sys.F(p));
            if (speed > seedSpeedThresh) continue;

            Vec2 r = p;
            if (!newtonSolveEquilibrium(sys, r)) continue;
            if (!box.contains(r)) continue;
            if (tooClose(r)) continue;

            roots.push_back(r);
        }
    }
    return roots;
}

}

renderer_2d.h
#pragma once
#include <glad/glad.h>
#include <glm/glm.hpp>
#include <vector>
#include <cstdint>

struct VertexPC {
    glm::vec2 pos;
    glm::vec3 color;
};

struct DrawRange {
    GLint first = 0;
    GLsizei count = 0;
};

class PolylineBatch {
public:
    GLuint vao = 0, vbo = 0;
    std::vector<VertexPC> cpuVertices;

```



```

std::vector<DrawRange> ranges;

void initGL() {
    glGenVertexArrays(1, &vao);
    glGenBuffers(1, &vbo);

    glBindVertexArray(vao);
    glBindBuffer(GL_ARRAY_BUFFER, vbo);

    glEnableVertexAttribArray(0);
    glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, sizeof(VertexPC),
(void*)offsetof(VertexPC, pos));

    glEnableVertexAttribArray(1);
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, sizeof(VertexPC),
(void*)offsetof(VertexPC, color));

    glBindVertexArray(0);
}

void clear() {
    cpuVertices.clear();
    ranges.clear();
}

void addPolylineVertices(const std::vector<VertexPC>& verts) {
    if (verts.size() < 2) return;
    DrawRange r;
    r.first = (GLint)cpuVertices.size();
    r.count = (GLsizei)verts.size();
    ranges.push_back(r);

    cpuVertices.insert(cpuVertices.end(), verts.begin(), verts.end());
}

void addPolyline(const std::vector<glm::vec2>& pts, glm::vec3 color) {
    if (pts.size() < 2) return;
    std::vector<VertexPC> v;
    v.reserve(pts.size());
    for (auto& p : pts) v.push_back({ p, color });
    addPolylineVertices(v);
}

void upload(GLenum usage = GL_DYNAMIC_DRAW) {
    glBindBuffer(GL_ARRAY_BUFFER, vbo);
    glBufferData(GL_ARRAY_BUFFER,
        cpuVertices.size() * sizeof(VertexPC),
        cpuVertices.empty() ? nullptr : cpuVertices.data(),
        usage);
}

void draw() const {
    glBindVertexArray(vao);
    for (auto& r : ranges) glDrawArrays(GL_LINE_STRIP, r.first, r.count);
    glBindVertexArray(0);
}

void destroy() {
    if (vbo) glDeleteBuffers(1, &vbo);
    if (vao) glDeleteVertexArrays(1, &vao);
    vao = vbo = 0;
}

```

```

    }
};

class LinesBatch {
public:
    GLuint vao = 0, vbo = 0;
    std::vector<VertexPC> cpuVertices;

    void initGL() {
        glGenVertexArrays(1, &vao);
        glGenBuffers(1, &vbo);

        glBindVertexArray(vao);
        glBindBuffer(GL_ARRAY_BUFFER, vbo);

        glEnableVertexAttribArray(0);
        glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, sizeof(VertexPC),
(void*)offsetof(VertexPC, pos));

        glEnableVertexAttribArray(1);
        glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, sizeof(VertexPC),
(void*)offsetof(VertexPC, color));

        glBindVertexArray(0);
    }

    void clear() { cpuVertices.clear(); }

    void addSegment(glm::vec2 a, glm::vec2 b, glm::vec3 c) {
        cpuVertices.push_back({ a,c });
        cpuVertices.push_back({ b,c });
    }

    void upload(GLenum usage = GL_DYNAMIC_DRAW) {
        glBindBuffer(GL_ARRAY_BUFFER, vbo);
        glBufferData(GL_ARRAY_BUFFER,
            cpuVertices.size() * sizeof(VertexPC),
            cpuVertices.empty() ? nullptr : cpuVertices.data(),
            usage);
    }

    void draw() const {
        glBindVertexArray(vao);
        glDrawArrays(GL_LINES, 0, (GLsizei)cpuVertices.size());
        glBindVertexArray(0);
    }

    void destroy() {
        if (vbo) glDeleteBuffers(1, &vbo);
        if (vao) glDeleteVertexArrays(1, &vao);
        vao = vbo = 0;
    }
};

```

shaders.h
#pragma once

```

static const char* kLineVS = R"GLSL(
#version 330 core
layout(location=0) in vec2 aPos;

```

```

layout(location=1) in vec3 aColor;

uniform mat4 uVP;

out vec3 vColor;
void main() {
    vColor = aColor;
    gl_Position = uVP * vec4(aPos, 0.0, 1.0);
}
)GLSL";

static const char* kLineFS = R"GLSL(
#version 330 core
in vec3 vColor;
out vec4 FragColor;
void main() {
    FragColor = vec4(vColor, 1.0);
}
)GLSL";

gl_utils.h
#pragma once
#include <glad/glad.h>
#include <string>
#include <iostream>

inline GLuint compileShader(GLenum type, const char* src) {
    GLuint sh = glCreateShader(type);
    glShaderSource(sh, 1, &src, nullptr);
    glCompileShader(sh);

    GLint ok = 0;
    glGetShaderiv(sh, GL_COMPILE_STATUS, &ok);
    if (!ok) {
        char log[2048];
        glGetShaderInfoLog(sh, 2048, nullptr, log);
        std::cerr << "Shader compile error:\n" << log << "\n";
    }
    return sh;
}

inline GLuint linkProgram(GLuint vs, GLuint fs) {
    GLuint prog = glCreateProgram();
    glAttachShader(prog, vs);
    glAttachShader(prog, fs);
    glLinkProgram(prog);

    GLint ok = 0;
    glGetProgramiv(prog, GL_LINK_STATUS, &ok);
    if (!ok) {
        char log[2048];
        glGetProgramInfoLog(prog, 2048, nullptr, log);
        std::cerr << "Program link error:\n" << log << "\n";
    }

    glDeleteShader(vs);
    glDeleteShader(fs);
    return prog;
}

```